Group Members: William Perryman, Raymond Hu, Yin Huang, Cian Teague, Venkat Penmetsa

Github Repo: [namyrrep/ParkEasy: Project for Group 9 Intermediate Software Design.]

Requirements:
- Each group member should show their 4 functional requirements aka use cases
- 1 Non-functional requirements: ie all pages must respond within 5 seconds
- Use Diagrams
- Class Diagrams for the entire system

Security – William Perryman

Functional Requirements:
1. Blacklist Management Page:
2. Real-Time Intruder Pop-ups if a plate that is scanned and is on the restricted list
3. Overstay Tracking System
4. Security Log Export that a user can download as a PDF

Non-Functional Requirements:
- If an alert is slow then it could miss something, alerts must be sent at a timely manner 2-5 seconds

Use Case 1: Add Vehicle to Blacklist
Primary Actor: Property Manager
Stakeholders and Interests:
- Security Administrator wants to easily manage the list of restricted vehicles to keep the system accurate and up to date.

Preconditions:
- The administrator is logged into the system and has access to the Blacklist Management Page.

Success Guarantee:
- The blacklist is updated and any changes are saved and reflected immediately in the system.

Main Success Scenario:
1. Administrator navigates to the Blacklist Management Page.
2. System displays all current blacklisted plates.
3. Administrator adds, edits, or removes a plate entry.
4. System validates the input.
5. System saves the changes and confirms with a success message.

Extensions:
- 4a. Invalid input data: System displays an error and prompts addministrator for fix
- 5a. Duplicate Error: System displays error message plate already in database

Special Requirements: None
Technology and Data Variation List:
- Plate entry must be done manually or imported from another source

Use Case 2: Real-Time Intruder Pop-up Alert
Primary Actor: Security Officer or Property Manager
Stakeholders and Interests:
- Security Officer wants to be immediately notified when a blacklisted vehicle is detected so they can respond in time.

Preconditions:
- The system is actively scanning license plates and the scanned plate exists on the blacklist.

Success Guarantee:
- A pop-up alert appears on the security dashboard within 2–5 seconds of the plate being scanned.

Main Success Scenario:
- Camera scans a vehicle's license plate.
- System checks the plate against the blacklist.
- System finds a match.
- System displays a real-time pop-up alert on the dashboard.
- Alert shows the plate number, timestamp, and camera location.
- Officer reviews and acknowledges the alert.

Extensions:
- 3a. Plate is not on the blacklist: No alert plate added to log
- 4a. Dashboard is not open: alert is queued and delivered within 2-5 seconds of detection

Special Requirements:
- Alert must be delivered within 2–5 seconds of detection to ensure a timely security response.

Technology and Data Variation List:
- Alert may be delivered as an on-screen modal or a web push notification depending on the platform.

Use Case 3:
Primary Actor: Overstay Tracking System
Primary Actor: System
Stakeholders and Interests:
- Security Officer wants to be notified of vehicles that have exceeded their allowed time so that appropriate action can be taken.

Preconditions:
- A vehicle entry has been logged and an allowed time limit is configured in the system

Success Guarantee:
- The system automatically flags the overstaying vehicle and notifies security personnel

Main Success Scenario:

1. System logs a vehicle plate and entry timestamp upon arrival.
2. System continuously monitors the elapsed time for all active entries.
3. System detects that a vehicle has exceeded the allowed time limit.
4. System marks the vehicle as "Overstay" on the tracking dashboard.
5. System sends a notification to the security officer with the plate number, location, and overstay duration.

Extensions:

- 3a. Vehicle has already exited, system closes the record
- 5a. No manager on site, notification is used

Requirements: None

Technology and Data Variation List:

- Need to be able to find license plate in database and store time stayed or when proposed exit time is

Use Case 4: Security Log Export as PDF

Primary Actor: Property Manager
Stakeholders and Interests:
- Property manager wants to download a formatted record of events for reporting and sharing purposes

Preconditions:
- The administrator is loggin in and security log data exists for selected date range.

Success Guarantee:
- A formatted PDF file containing the filtered security log is successfully downloaded to the device it is being used on

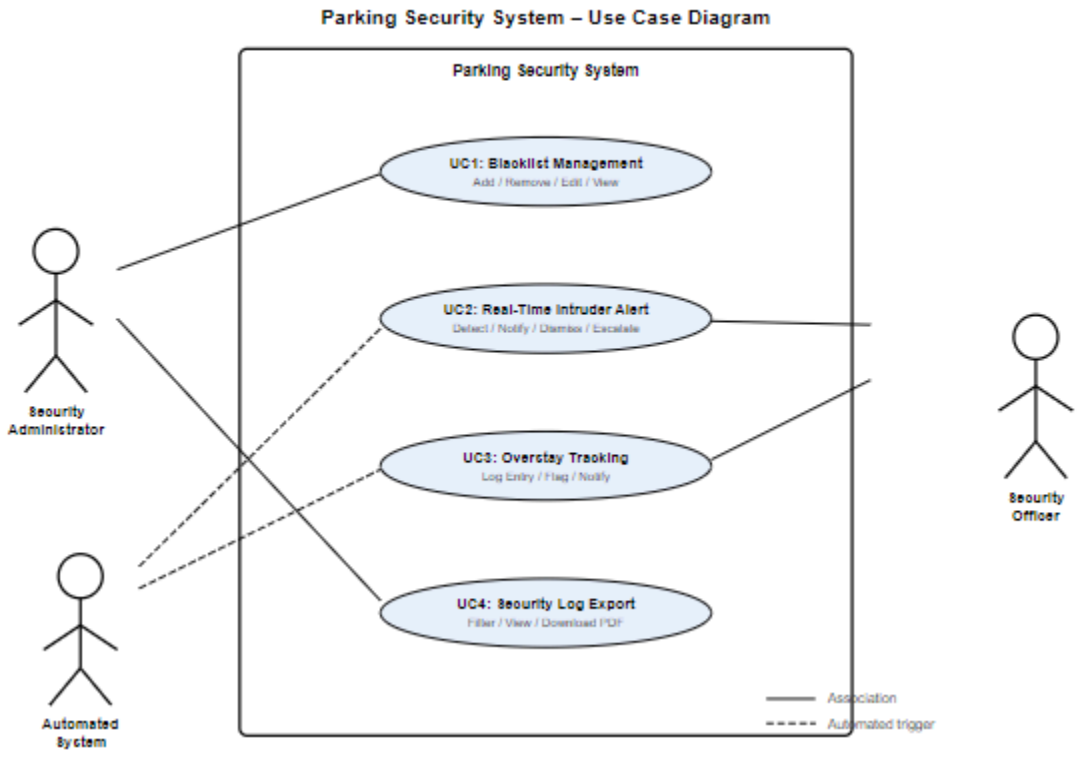
Main Success Scenario:
1. Administrator navigates to the Security Log section.
2. Administrator selects a date range and optionally filters by event type.
3. System retrieves and displays the matching log entries on screen.
4. Administrator reviews the log and clicks "Download as PDF."
5. System generates the PDF and initiates the file download.

Extensions
- 3a. No entries exist for a selected range, system displays No records found for a selected period
- 5a. PDF generation fails, system displays an error message and prompts a retry

Special Requirements: (None)
Technology and Data Variation List:
- PDF may be generated client side or server side, data selection may use calender picker or manual text input

Supplementary Specifications / Non Functional Requirements

Preformance time: The system must detect a blacklisted plate and deliver an alert before 5 seconds elapse, or it should send an alert



Admin/Auth – Cian Teague

Functional Requirements:

- The system will ensure that only authorized users are able to access appropriate systems/devices
- The system will verify that these users are the only ones allowed to interact with their respective devices and information
- The system will provide information about activity on these devices only to authorized users
- The system will offer secure access to those sensors and cameras from the internet, allowing users to access our web application dashboard from any device

Non-Functional Requirements:

- User verification must be able to happen within 5 seconds

Use Case UC1: Authenticate User Access

- **Primary Actor:** Authorized User
- **Stakeholders and Interests:**
    - **Authorized User:** Needs quick and reliable access to both their account and dashboard.
    - **System Administrator:** Wants to ensure that unauthorized users are unable to access the system.
- **Preconditions:**
    - The user has a registered account within the system.
- **Success Guarantee:**
    - The user is successfully authenticated and then granted access to the web application dashboard.
- **Main Success Scenario:**
    1. The user navigates to the web application login screen.
    2. The user enters their credentials (e.g., username and password).
    3. The system then validates the credentials against its database.
    4. The system grants access to the user and redirects them to their dashboard.
- **Extensions:**

  3a. Invalid credentials:
    1. The system alerts the user that the login details were invalid.
    2. The user is then prompted to try again.
- **Special Requirements:**
    - User verification must be able to happen within 5 seconds, since any longer could inhibit user's reaction to serious alerts.
- **Technology and Data Variation List:**
    - User credentials may be entered via a web browser on either a desktop, laptop, or mobile device.

Use Case UC2: Authorize Device Interaction

- **Primary Actor:** Authorized User
- **Stakeholders and Interests:**
    - **Authorized User:** Needs to be able to interact with and control their devices in a secure manner.
    - **System Administrator:** Needs to verify that permissions to these devices are strictly enforced.
- **Preconditions:** The user is both authenticated and logged into the dashboard.
- **Success Guarantee:** The system verifies the user's permissions and which devices they have permissions to use, and successfully processes the device interaction command.
- **Main Success Scenario:**
    1. The user selects a specific device (e.g., gate, camera, sensors, etc.) from the dashboard.
    2. The user submits the request to interact with the chosen device.

3. The system then verifies that the user is authorized to interact with the device.
4. Once authorization is verified, the system executes the command on the device.
5. The system then confirms the successful execution of this user's request to interact with the device.

- **Extensions:**

  3a. Unauthorized interaction attempt:
    1. While verifying if the user is authorized to interact with the requested device, the system detects that the user does not hold permissions for this device.
    2. The system then denies this request and keeps track of the unauthorized attempt.
    3. The system then alerts the user with an access denied message.
- **Special Requirements:** Both authorization checks and user verification must be able to happen within 5 seconds.
- **Technology and Data Variation List:** Device interactions will happen over the internet, therefore allowing control from any device.

Use Case UC3: View Device Activity Logs

- **Primary Actor:** Authorized User
- **Stakeholders and Interests:**
  - **Authorized User:** Needs to be able to monitor both the activity and status of their devices.
  - **System Administrator:** Needs to ensure both privacy and data isolation between different users' data.
- **Preconditions:** The user is not only logged in, but also the activity data for the user's device exists.
- **Success Guarantee:** The system retrieves and displays the user's activity logs for only their verified devices.
- **Main Success Scenario:**
  1. The user requests to see the activity history or logs.
  2. The system verifies that the user is authorized to do so.
  3. The system then scans the database for the activity records tied to the user's devices.
  4. The system then displays the appropriate information to the user.
- **Extensions:**

  2a. Session Expiration
    1. The system detects that the user has been inactive for too long, deeming their authentication session expired.
    2. The system then redirects the user to the login page.

  3a. No activity data found
    1. The system doesn't find any activity for the user's devices.
    2. The system then alerts the user of this with a "no recent activity" message.
- **Special Requirements:** Both user verification and data retrieval must respond within 5 seconds.

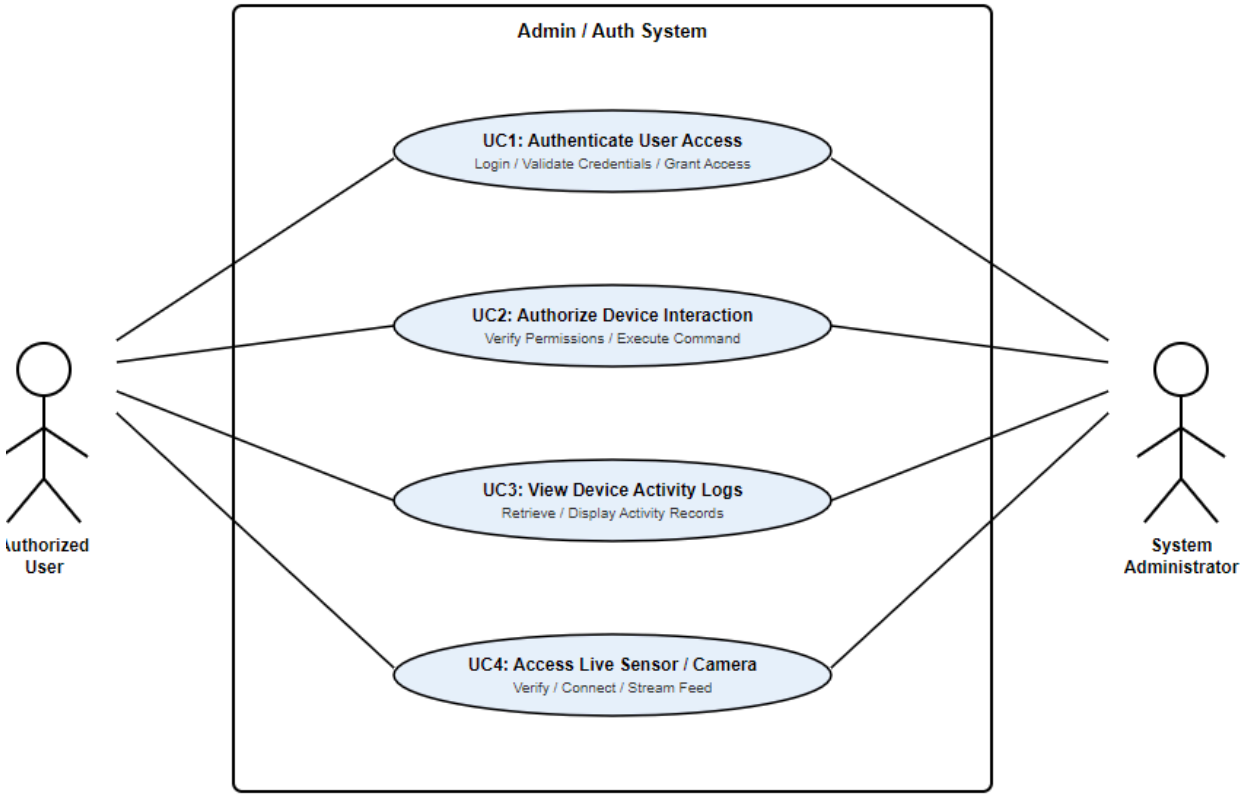
- **Technology and Data Variation List:** Activity logs are able to be viewed from any internet-connected device via the web application dashboard.

Use Case UC4: Access Live Sensor

- **Primary Actor:** Authorized User
- **Stakeholders and Interests:**
    - **Authorized User:** Needs reliable and secure remote access to live camera feeds as well as sensor data.
    - **System Administrator:** Needs to ensure that hardware feeds are secure from unauthorized users.
- **Preconditions:** The user is logged in while their cameras and sensors are operational
- **Success Guarantee:** A secure connection is verified, and the user is able to view either their live camera or sensor feed from their dashboard.
- **Main Success Scenario:**
    1. The user navigates tot he camera/sensor live view section of the web application dashboard.
    2. The system then verifies that the user is permitted to access these cameras or sensors.
    3. The system then establishes a secure connection from the hardware to the user's dashboard.
    4. The user is then able to view the real-time data/video feed.
- **Extensions:**

  3a. Hardware is offline:
    1. The system attempts to connect the user to their requested camera/sensor, but receives no response.
    2. The system then alerts the user of this issue, displaying an "Offline/Unreachable" status message.
- **Special Requirements:** The verification for the user to access the feed must happen within 5 seconds.
- **Technology and Data Variation List:** Both video and sensor streams can be accessed with an internet-connected device, and is able to be rendered to desktops, laptops, or mobile devices.

Admin / Authentication System – Use Case Diagram



Payments – Raymond Hu

Use Case 1: Generate QR Ticket

- **Primary Actor:** Driver
- **Stakeholders and Interests:**
  - **Driver:** Wants quick ticket generation to enter the parking space.
  - **Parking Operator:** Wants each session to be properly recorded and linked to a unique QR code.
  - **System Administrator:** Wants reliable session tracking.
- **Preconditions:**
  - Driver requests entry into the parking system.
  - System is operational.
- **Success guarantee:**
  - A unique QR ticket is generated.
  - A session record is created and stored.
  - The QR code is linked to the session record.
  - The session record includes entry time, location, and vehicle identifier.

- **Main success scenario:**
  1. The driver requests a parking ticket.
  2. The system creates a new parking session and records entry time, location, and vehicle identifier.
  3. The system generates a unique QR code and links it to the session record.
  4. The system displays or prints the QR ticket.
- **Extensions:**

  2a. Session Creation Failure

  1. The system fails to create a session record.
  2. The system displays an error message and sends an error report.
  3. The driver may retry the request.

  3a. QR Code Generation Failure

  1. The system fails to generate a QR code.
  2. The system displays an error message and sends an error report.
  3. Session creation is canceled.
  4. The driver may retry the request.
- **Special requirements:**
  - QR ticket generation must be complete within 2 seconds under most scenarios
- **Technology and data variation list:**
  - The QR code may be displayed on a mobile device or printed at a kiosk.
  - The vehicle identifier may be manually entered or captured via license plate recognition.

Use Case 2: Retrieve Previous Session Information

- **Primary Actor:** Driver
- **Stakeholders and Interests:**
  - **Driver:** Wants to verify the accuracy of the payment.
  - **Parking Operator:** Wants transparency and accurate record retrieval.
- **Preconditions:**
  - A previous session exists in the system.
  - The driver has a valid QR code.
- **Success guarantee:**
  - The system retrieves and displays the correct session information.
- **Main success scenario:**
  1. The driver scans or enters a previous QR code.

2. The system searches for the associated session in the database.
3. The system retrieves session details from the database.
4. The system displays entry time, location, duration, and payment status.

- **Extensions:**

  2a. Session Not Found

    1. The system does not find a matching session record and displays "Session not found."
    2. The driver may attempt to re-enter the QR code.

  2b. Invalid QR Code

    1. The system determines that the QR code is invalid and then displays an error message.

  2c. System Unavailable

    1. The system cannot access the session database and then displays an error message
    2. The driver may try again or contact the parking operator.

- **Special requirements:**
  - Retrieval must occur within 2 seconds in most cases.
- **Technology and data variation list:**
  - QR code may be scanned via the mobile app or at akiosk.
  - Session data may be stored in a local or cloud database.

Use Case 3: Calculate Parking Fees

- **Primary Actor:** Driver and System Database
- **Stakeholders and Interests:**
  - **Driver:** Wants an accurate, real-time fee calculation.
  - **Parking Operator:** Wants the correct fee configuration and reliable revenue collection.
- **Preconditions:**
  - A valid parking session exists.
  - Pricing configurations are defined in the system.
- **Success guarantee:**
  - The system calculates the correct parking fee based on configured rules.
- **Main success scenario:**

1. The driver initiates payment or requests that the fee be displayed.
2. The system retrieves the session record and finds the parking duration.
3. The system applies pricing configurations.

4. The system calculates the total fee and displays the calculated amount.
5. The fee updates in real time if the duration changes.

- **Extensions:**

  2a. Session Not Found

  1. The system cannot locate the session record and displays an error message.

  3a. Pricing Configuration Error

  1. The system detects missing or invalid pricing rules and displays a configuration error message.
  2. Fee calculation cannot proceed.

- **Special requirements:**
  - The fee calculation must occur within 2 seconds in most cases.
  - Calculation must update in real time.
- **Technology and data variation list:**
  - Pricing may vary by location, time of day, or special events.
  - Configuration rules may be updated by the parking operator.

Use Case 4: Process Payment and Mark QR as Paid

- **Primary Actor:** Driver
- **Stakeholders and Interests:**
  - **Driver:** Wants a secure and fast transaction.
  - **Parking Operator:** Wants confirmed payment and prevention of QR reuse.
  - **Payment Provider:** Wants secure transaction processing.
- **Preconditions:**
  - A valid QR code exists.
  - The fee has been calculated.
  - The QR code has not already been marked as paid.
- **Success guarantee:**
  - Payment is successfully processed.
  - The QR code is marked as paid.
  - The QR code cannot be reused unless a new session is created.
  - Payment confirmation is displayed to the driver.
- **Main success scenario:**

  1. The driver scans the QR code to pay.
  2. The system retrieves the session and the fee information.
  3. The system displays the total amount due.

4. The driver selects their payment method.
5. The system processes the transaction.
6. The payment provider confirms the transaction is valid.
7. The system marks the QR code as paid.
8. The system confirms that the payment is complete to the driver.

- **Extensions:**

  2a. Invalid QR Code

  1. The system determines that the QR code is invalid or not found.
  2. The system displays an error message.

  2b. QR Code Already Paid

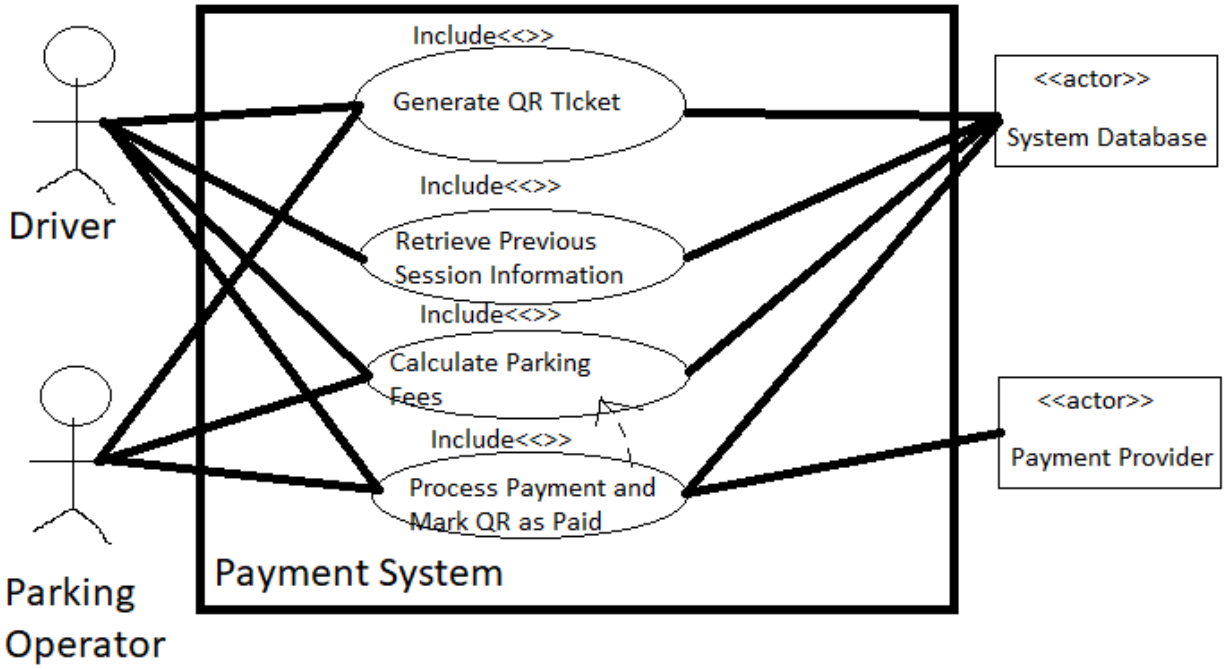
  1. The system detects that the QR code has already been marked as paid.
  2. The system displays a message indicating that the payment was already completed.
  3. The transaction is canceled.

  7a. Payment Failure

  1. Payment provider rejects the transaction
  2. The system displays a payment failure message and reason.
  3. The driver may retry the payment or choose another method.

- **Special requirements:**
  - Payment processing and confirmation must be completed within 2 seconds in most cases.
  - The system must prevent the reuse of paid QR codes.
- **Technology and data variation list:**
  - Payment methods: credit card, debit card, mobile wallet.
  - The QR code can be scanned through a mobile device or a kiosk.

Use Case Diagram:



AI Recognition System – Venkat Penmetsa

Use Case 1 UC1: Account Registration

- **Primary Actor: User**
- **Stakeholders and Interests:**
  - **User: wants to access account to see matches that their vehicle has**
  - **System admin: Wants to be able to trace actions and matches for each vehicle for each user**
- **Precondition: The user does not have an account**
- **Success guarantee: The user exists and can log in to their account**
- **Main Success Scenario:**
  - **User opens system and creates account**
  - **User enters in username and password upon being prompted by the system**
  - **System validates that the information entered is correct and traces an account session by the user**

    - User is then able to access the main page and continue on to use the other features.
- Extensions:
    - 2a. Password or username entered is incorrect. User is prompted to enter it again.
    - 3a. Account with same username or email is already being used. User will be asked to select a new username/email
    - 3b. System does not properly validate information-the system possibly crashes. User will then be asked to create an account at another time.
- Special Requirements:
    - User must accept a verification email to verify and start using the account
- Technology/Data Variations:
    - SSO similar to that of VT Canvas could be used

Use Case 2 UC2 Vehicle and License Plate Matching with User:

- Primary Actor: User
- Stakeholder and Interests:
    - User: License Plate should be connected to the correct vehicle
    - System: Plate is correctly matched to the vehicle
    - Admin: Wants the matching algorithm to work correctly
- Preconditions: User has an account and has entered into account
- Success Guarantee: Vehicle matching algorithm is used to correctly match the vehicle to the correct license plate
- Main Success Scenario:
    - User adds a vehicle on their account
    - System asks user to enter their license plate number
    - User enters their license plate number
    - System validates the license plate number using database search
    - System stores a vehicle that is mapped to the license plate which is linked to the user
    - System makes sure that the user plate has been linked to the user
- Extensions:
    - 3a. Plate number entered is incorrect due to incorrect format. User is asked to enter plate number again.
    - 4a. Plate is already linked to another user. System asks user to enter a different plate number.

    - **5a. There is an issue with the database and the system cannot link the license plate to the user. Error message will be displayed.**
- **Special Requirements:**
    - **The plate text will be formatted as a string with just letters and no punctuation**
- **Technology/Data Variations:**
    - **User can have the option to enter the state their license plate is added to reduce confusion**

Use Case 3 UC3 Real-Time License Plate Recognition:

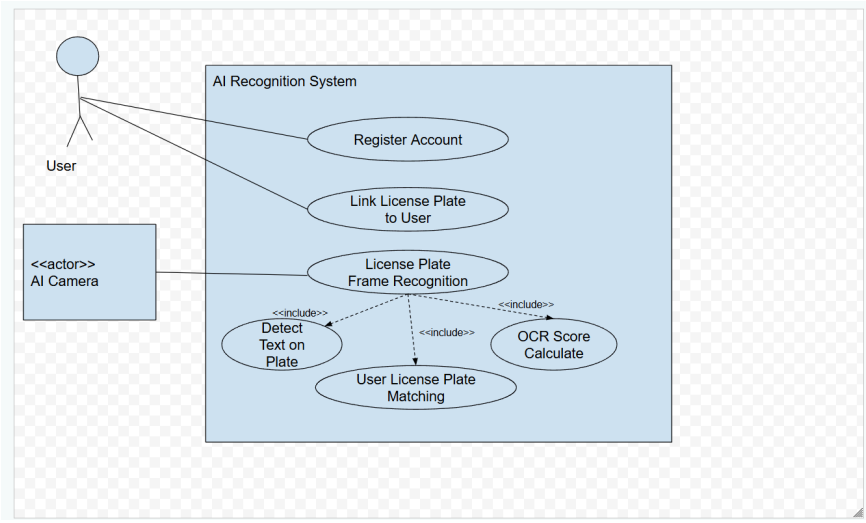
- **Primary Actor: Camera and AI Recognition System**
- **Stakeholders and Interests:**
    - **System Admin: Wants accurate detection**
    - **User: Wants license plate to be recognized correctly**
    - **Privacy and Security officials: does not want license plate data to be used incorrectly or unlawfully**
- **Preconditions:**
    - **System can access cameras' images**
    - **Camera is live and working**
- **Success Guarantee:**
    - **It is recorded the confidence of the frames being matched along with when they were matched. This is known as a Recognition Event.**
- **Main Success Scenario:**
    - **The camera sends the frames to the system**
    - **The system tries to identify if there is a license plate in the frame**
    - **System uses OCR algorithm to get the text from the license plate if it is there**
    - **System creates a Recognition Event with the text for the license plate, the confidence score from the OCR, and the time when the plate was identified**
    - **System stores the event in the database for vehicle and user matching**
- **Extensions:**
    - **1a. Camera is not working properly and the frame is not properly collected. System will stop detecting and send error message to the log.**
    - **2a. A plate or the text on the plate is not detected and a failure Recognition Event is recorded and sent to the system**

    - **3a. The confidence level does not meet the threshold. System will put a warning flag on this Recognition Event.**
    - **5a. Database crashes or has a bug. The system will stop running and an error message will be sent to the log.**
- **Special Requirements:**
    - **Only store the part of the image that has the license plate and the text.**
- **Technology and Data Variations:**
    - **System may have to cancel noise or alter frames to meet memory constraints**

Use Case 4 UC4: Matching Recognition Events with User

- **Primary Actor: System**
- **Stakeholders and Interests:**
    - **User: Events are correctly matched to user and their vehicle**
    - **System Admin: Matching logic should be correct**
- **Preconditions:**
    - **There is an existing Recognition Event that will be analyzed**
- **Success Guarantee:**
    - **Event is either matched to a user or it is not matched to a user and left in the database**
- **Main Success Scenario:**
    - **System receives a new Recognition Event**
    - **The text for the license plate is turned into a formatted string that just has uppercase letters**
    - **System searches user database to find a vehicle that has the same license plate number**
    - **If there is a match, there there is a link put in place between the user, the vehicle, and the Recognition Event**
    - **The event is marked as a match by the system**
    - **The system will then allow the user to see the match that they got.**
- **Extensions:**
    - **3a. If there is no match found between the Recognition Event and any user, it is sent to the admin for review**
    - **6a System is not notified of match and hence, event does not get changed to a matched event**
- **Special Requirements:**

    - **N/A**
- **Technology and Data Variations:**
    - **The confidence score will be output once the plate is read and confirmed.**



- **Supplementary Specifications:**
    - **For each frame, the text and the OCR should be completed within a certain threshold of a small number of seconds**
    - **All user and administrator requests and actions must be backed by authentication, and the data in the system database must be encrypted**

Data/Brain - Yin Huang

Use Case 1: Real-time Occupancy Synchronization

Primary Actor: System / Hardware Sensors
Stakeholders and Interests:
- **Property Owner: Wants to see the exact number of cars in the lot to manage space efficiently.**

- **Driver: Needs to know if there are available spots before they arrive.**

Preconditions: Hardware sensors and cameras are online and connected to the cloud API.

- **Success Guarantee: The database and dashboard show the same, correct number of occupied spots.**
- **Main Success Scenario:**
    - **Sensor detects a vehicle entering or leaving a parking spot.**
    - **The system receives this data through the Hardware Link API.**
    - **Data/Brain module validates the incoming data to ensure it's not a false positive.**
    - **System updates the "Occupancy" count in the database immediately.**
    - **The Web Dashboard refreshes to show the updated vacancy to the user.**
- **Extensions:**
    - **3a. Data Conflict: If sensors and cameras show different results, the system marks the spot for manual review.**
- **Special Requirements: Syncing should happen in real-time to avoid "double booking" a spot.**

Use Case 2: Apply Dynamic Pricing & Billing Logic

- **Primary Actor: Parking Operator**
- **Stakeholders and Interests:**
    - **Parking Operator: Wants to increase revenue during busy hours (like game days).**
- **Preconditions: The operator is logged into the admin dashboard with "Pricing Control" permissions.**
- **Success Guarantee: All new sessions follow the new pricing rules correctly.**
- **Main Success Scenario:**
    - **Operator sets a "Peak Hour" multiplier (e.g., 1.5x) for a specific time range.**
    - **System saves these rules into the billing configuration database.**
    - **When a driver requests a ticket or pays, the system checks the current time against these rules.**
    - **The system calculates the final fee based on the entry/exit duration and the active rate.**
    - **The updated price is sent to the Payment module for processing.**
- **Extensions:**
    - **2a. Invalid Pricing Input: System warns the operator if the price is set to zero or an impossible value.**

Use Case 3: Generate Predictive Analytics for Peak Hours

- **Primary Actor: Property Manager**
- **Stakeholders and Interests:**
    1. **Manager: Wants to know when the lot is usually full to plan for staffing or**

maintenance.

- **Preconditions: The system has at least one week of historical parking data.**
- **Success Guarantee: A clear chart showing predicted busy times is displayed on the dashboard.**
- **Main Success Scenario:**
    1. **Manager navigates to the "Analytics" tab on the dashboard.**
    2. **The system pulls historical occupancy data from the database.**
    3. **The Data/Brain algorithm processes this data to find patterns.**
    4. **System generates a "Busy-Hour Chart" for the upcoming 24 hours.**
    5. **Manager reviews the data to make business decisions.**

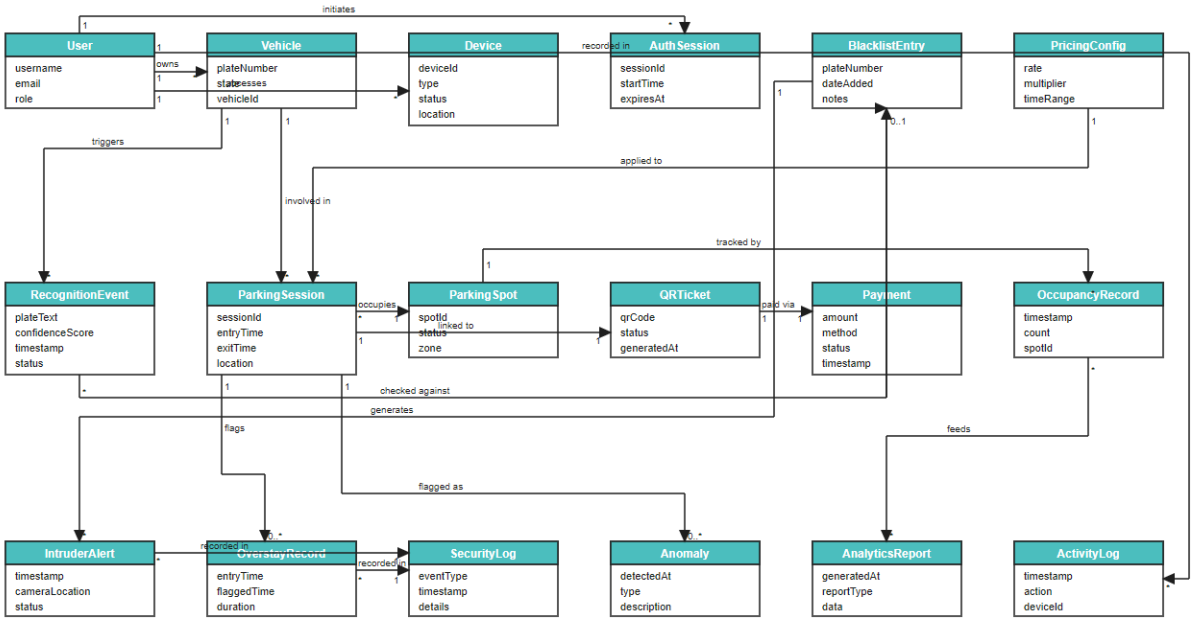
Use Case 4: Data Integrity & Anomaly Detection

- **Primary Actor: System**
- **Stakeholders and Interests:**
    1. **Security Officer: Needs to know if a car has stayed way longer than paid for.**
- **Preconditions: The database contains active parking sessions.**
- **Success Guarantee: Any data errors or suspicious activity are flagged and logged.**
- **Main Success Scenario:**
    1. **The system runs a background scan of all active sessions every 10 minutes.**
    2. **It compares entry times with current occupancy status.**
    3. **The system identifies a "ghost car" (a car that never exited according to the sensor but isn't on camera).**
    4. **The system marks this record as an "Anomaly."**
    5. **An alert is sent to the Security module to check the lot.**

Supplementary Specification (Non-Functional Requirement)

- **Data Consistency & Concurrency Control: Our system must handle multiple cars entering or exiting at different gates at the same time. To ensure we don't report more free spots than we actually have, the database must use atomic updates for the occupancy count. Under a load of 50 concurrent requests per second, the data update must complete within 500ms to keep the dashboard accurate.**

Class diagram:


initiates
User
username
email
role
owns
Vehicle
plateNumber
vehicleId
Device
deviceId
type
status
location
recorded in
AuthSession
sessionId
startTime
expiresAt
BlacklistEntry
plateNumber
dateAdded
notes
PricingConfig
rate
multiplier
timeRange
triggers
applied to
involved in
tracked by
RecognitionEvent
plateText
confidenceScore
timestamp
status
ParkingSession
sessionId
entryTime
exitTime
location
occupies
ParkingSpot
spotId
status
zone
linked to
QRTicket
qrCode
status
generatedAt
paid via
Payment
amount
method
status
timestamp
OccupancyRecord
timestamp
count
spotId
checked against
generates
flags
feeds
flagged as
IntruderAlert
timestamp
cameraLocation
status
recorded in
OverstayRecord
entryTime
flaggedTime
duration
recorded
SecurityLog
eventType
timestamp
details
Anomaly
detectedAt
type
description
AnalyticsReport
generatedAt
reportType
data
ActivityLog
timestamp
action
deviceId